AI with Rav

Learn AI the way you actually think.

DAY 2 of 30

OPEN SOURCE CONTRIBUTION

The Bug the Maintainer Filed Himself

WHAT YOU'LL LEARN TODAY

- How to read a bug report written by an expert
- Why one image broke and a similar one did not
- How to change a function without breaking its callers
- Taking a maintainer's suggestion the right way

The Bug the Maintainer Filed Himself

> In One Line: pypdf could not open a 4-bit colour image. The person who filed the bug was the maintainer himself — which made it the safest issue on the tracker to pick up.

Why this one was worth picking

Day 1's bug came from a user. This one came from **stefan6419846** — the pypdf maintainer.

That is a signal worth learning to read. When a maintainer files an issue on their own project, three things are already true before you write any code:

- The bug is **real** — nobody has to be convinced it exists
- The **scope is agreed** — the person who will review your PR already decided this should be fixed
- Nobody is **secretly working on it** — if they were, they would have said so in their own issue

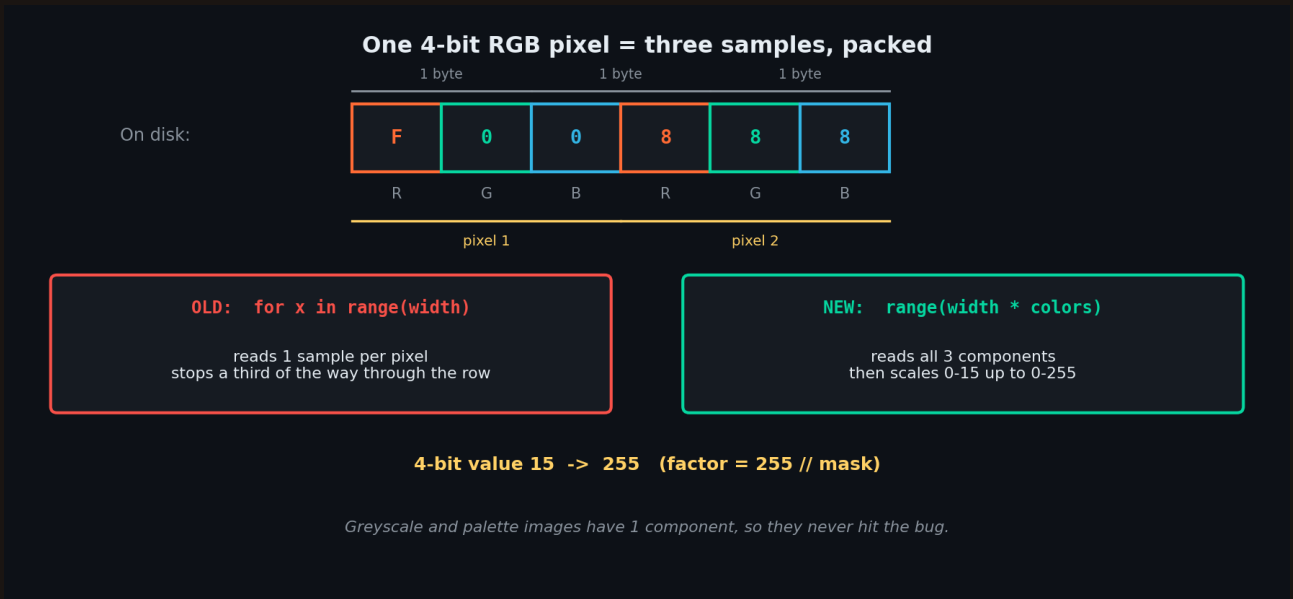
Most of what kills a first PR is disagreement about whether the bug matters. Here, that argument was over before I arrived.

The bug

A PDF containing an image with **4 bits per component** in the **/DeviceRGB** colour space would not extract. It failed inside Pillow with:

```
ValueError: unrecognized image mode 4bits
```

Not a wrong-looking image. A crash, with an error mentioning a "mode" no user has ever heard of.



A 4-bit RGB pixel is three samples packed into one and a half bytes. The old code read one sample per pixel and stopped.

What "4 bits per component" means

Normally each colour component gets a full byte: red 0–255, green 0–255, blue 0–255. Three bytes per pixel.

A 4-bit image gives each component only **4 bits** — a value from 0 to 15. Two components fit in a single byte. It is a compression trick: a quarter of the file size when the image does not need full colour depth.

To display it, you must do two things:

- **Unpack** — pull the 4-bit values out of the packed bytes
- **Scale** — stretch each value from the 0–15 range up to 0–255, so 15 becomes 255, not a nearly-black 15

Why it broke

pypdf did have unpacking code. It made one assumption:

```
# one sample per pixel
buffer_size = size[0] * size[1]
...
for x in range(size[0]):
    byte_buffer[x + y * size[0]] = (data[data_index] >> bit) & mask
```

size[0] is the width in **pixels**. That is correct for greyscale or a palette image, where one pixel is one sample.

For RGB, one pixel is **three** samples. So the loop read a third of the row, left the rest packed, and never scaled anything. Then the mode string **"4bits"** was handed to Pillow, which had no idea what it meant.

> Note how the bug hid: 4-bit greyscale and 4-bit palette images worked perfectly. Only RGB broke. A wrong assumption that is right most of the time is the hardest kind to spot.

The fix

Two new parameters on **bits2byte**, both defaulted so every existing caller behaves exactly as before:

```
def bits2byte(
    data: bytes,
    size: tuple[int, int],
    bits: int,
    colors: int = 1,      # components per pixel
    scale: bool = False,  # stretch to 0-255?
) -> bytes:
    samples_per_row = size[0] * colors
    ...
    factor = 255 // mask if scale and mask else 1
```

colors=3 makes it unpack the full row. **scale=True** stretches each sample to the full range. And **/DeviceRGB** now maps to mode **"RGB"** instead of **"P"**.

> Defaults of colors=1, scale=False mean the palette path is byte-for-byte unchanged. That is what let a reviewer say yes quickly — the risky part of the change is opt-in.

The lesson in that signature

When you need to change a shared function, you have two options.

Approach	What the reviewer must do
Change the behaviour for everyone	Verify every existing caller still works
Add opt-in parameters with safe defaults	Verify only the new path

The second one is not a trick — it is genuinely lower risk. Existing behaviour is preserved *by construction*, not by your promise that you checked.

How I confirmed it before writing code

The issue came with a PDF attached and four lines of Python. That is a gift, and the first thing to do with it is run it.

- Install the **exact version** in the report — **pypdf==6.14.2**, not whatever you already have
- Run the reproducer **unchanged** and confirm you get the same error
- Then run it against a **fresh clone** of **main**, in case it was already fixed

That third step is the one people skip, and it is the one that saves days. An issue being open does not mean the bug is still there — plenty of issues are fixed by an unrelated change and never closed.

> I have lost days to bugs that no longer existed. Now it is the first thing I check, before reading a single line of the implementation. Reproduce on main or walk away.

Finding the line

Once it reproduced, the error message did most of the work. **unrecognized image mode 4bits** is a Pillow error, so the question becomes: where does pypdf hand a mode string to Pillow?

Searching for the string **"4bits"** in the codebase leads straight to the low-bit branch. From there it is a matter of reading what the branch assumes — and **size[0]** being used as a sample count is visible in about two minutes once you are looking at the right twenty lines.

> Search for the literal text in the error message, not for what you think the feature is called. The error string is a unique token that exists in exactly one place. It is the fastest path into an unfamiliar codebase.

The review

The maintainer left one suggestion. I had written the bit-depth extraction one way; he proposed:

```
bits = int(mode[0])
```

Cleaner, and safe because the only low-bit modes that exist are **2bits** and **4bits**, so the first character is always the answer.

I took it. That is worth saying plainly: **when a maintainer suggests something simpler and it is correct, take it.** Do not defend your version because it is yours. He knows the codebase; you have read it for a week.

> CI went red on my PR — and it was not my fault. The download-fixture tests were failing on a network error. I said so, noted that my own test runs offline, and offered to rebase. Do not silently force-push at red CI; explain what you are seeing.

The test

```
# a 2x2 4-bit /DeviceRGB image, built in the test - no fixture file
assert img.image.mode == "RGB"
assert img.image.getpixel((0, 0)) == (255, 0, 0)
```

Constructed in the test itself, so it runs offline and fails on **main** with the exact **ValueError** from the issue.

What to copy

- **Read the issue tracker for maintainer-filed issues.** They are pre-approved work. Filter by author.
- **When touching shared code, make new behaviour opt-in.** Defaults that preserve the old path are your best argument.
- **Take the suggestion.** A maintainer's one-line simplification is a gift, not a criticism.
- **Say something when CI fails.** Silence looks like you did not notice.

What is next

This fix was correct — and incomplete. It only worked for compressed images. The maintainer noticed, told me so, and the next video is what happened then.